

CS121 Data Structures in C

14 *Graph*

Outline

- Graph traversal
 - ♦ Breadth-First Search
 - ♦ Depth-First Search
- Spanning tree
 - ♦ BFS tree
 - ♦ DFS tree

Graph Traversal

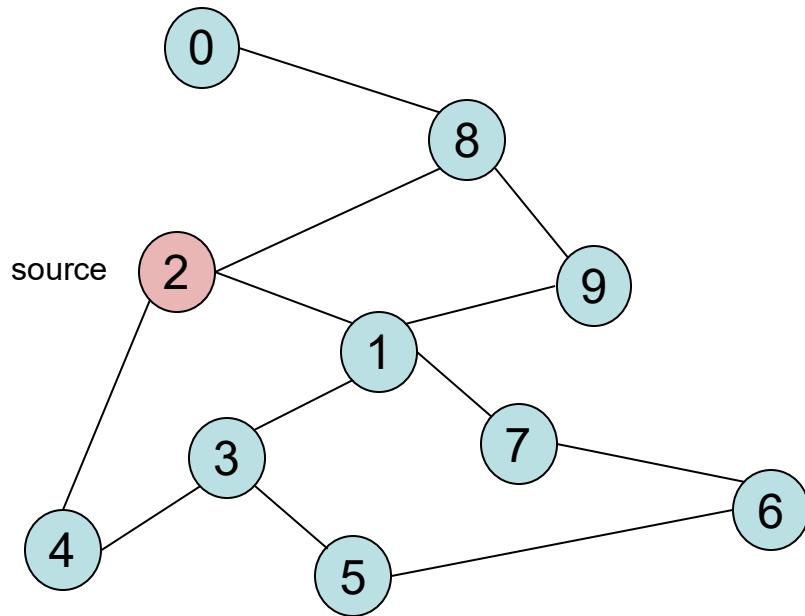
- Application example
 - ♦ Given a graph representation and a vertex **s** in the graph.
 - ♦ Find all paths from **s** to other vertices, e.g. is there a path between **s** and **v**?, if so, what is the path between **s** and **v**?
- Two common graph traversal algorithms
 - ♦ Breadth-First Search (BFS)
 - ♦ Depth-First Search (DFS)
- Spanning tree
 - ♦ A tree that contains all of the vertices in the graph.
 - ♦ DFS tree
 - ♦ BFS tree

Breadth-First Search

Using the queue to implement: need FIFO

- Begin by enqueueing the starting vertex, A, in the queue;
- After processing it, enqueueing all of its adjacent vertices in the queue;
- Loop, dequeuing the queue and processing the vertex from the front of the queue.
 - ♦ After processing the vertex, we place all of its adjacent vertices into the queue. We are then ready for the next step.
- When the queue is empty, the traversal is complete.

BFS Example



$Q = \{ \}$

Initialize **Q** to be empty

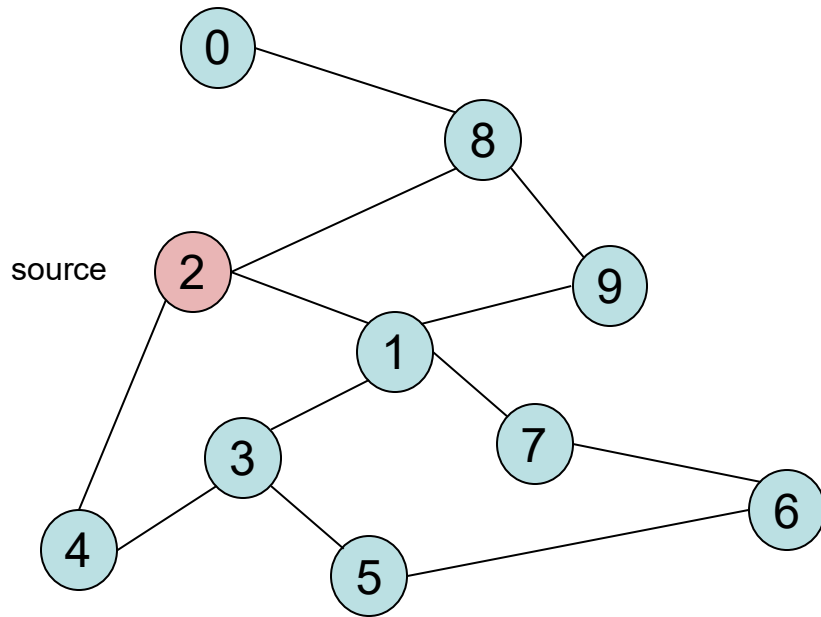
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	F
1	F
2	F
3	F
4	F
5	F
6	F
7	F
8	F
9	F

Initialize visited
table (all False)



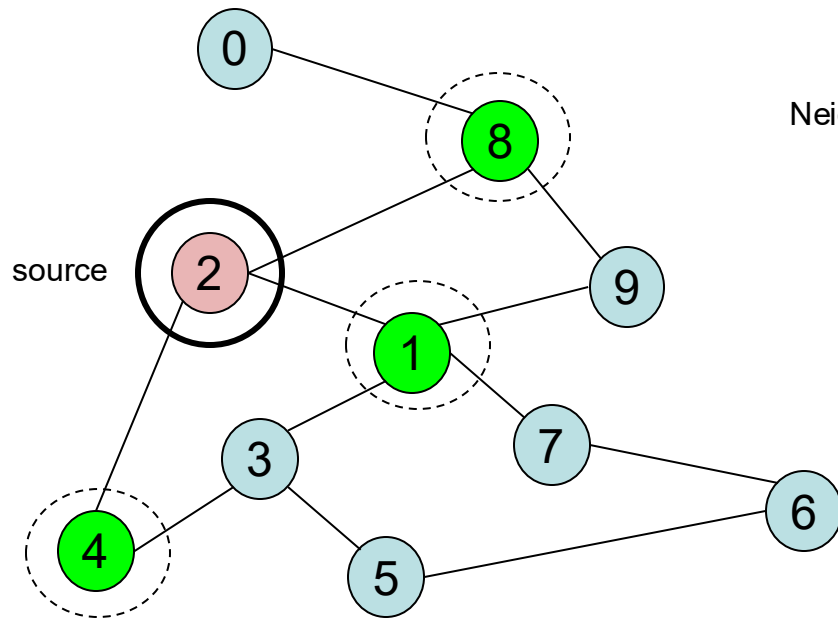
$Q = \{ 2 \}$

Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	F
1	F
2	T
3	F
4	F
5	F
6	F
7	F
8	F
9	F



$Q = \{2\} \rightarrow Q = \{ 8, 1, 4 \}$

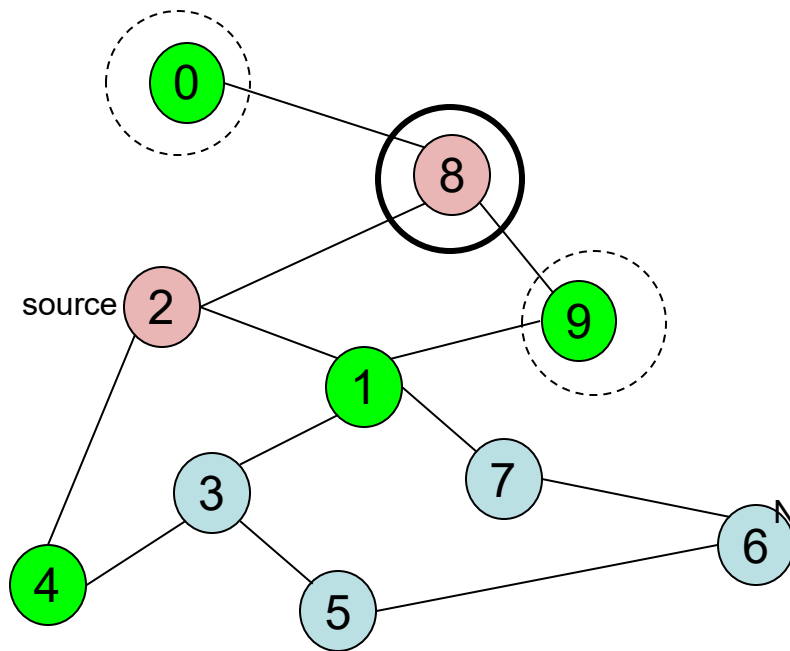
Adjacency List

Neighbors →

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	F
1	T
2	T
3	F
4	T
5	F
6	F
7	F
8	T
9	F



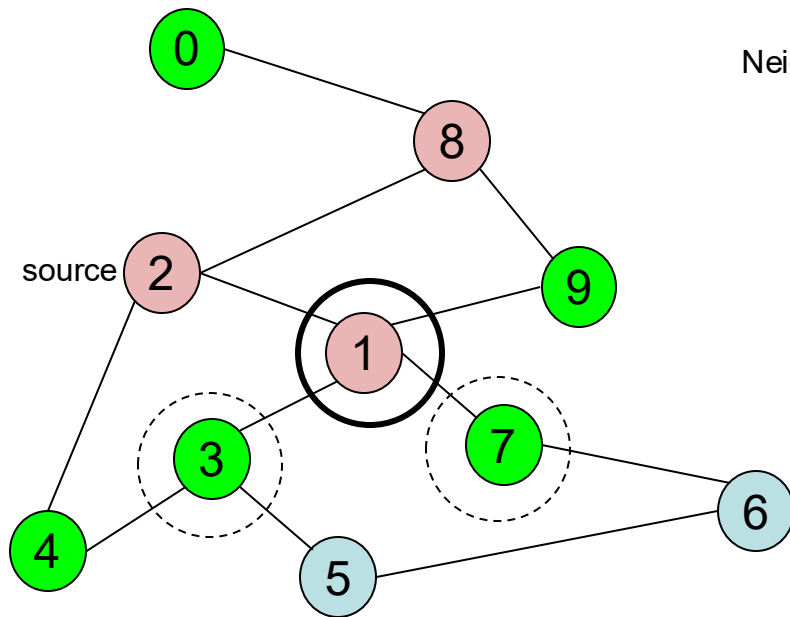
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	F
4	T
5	F
6	F
7	F
8	T
9	T

$\mathbf{Q} = \{ 8, 1, 4 \} \rightarrow \mathbf{Q} = \{ 1, 4, 0, 9 \}$



Neighbors →

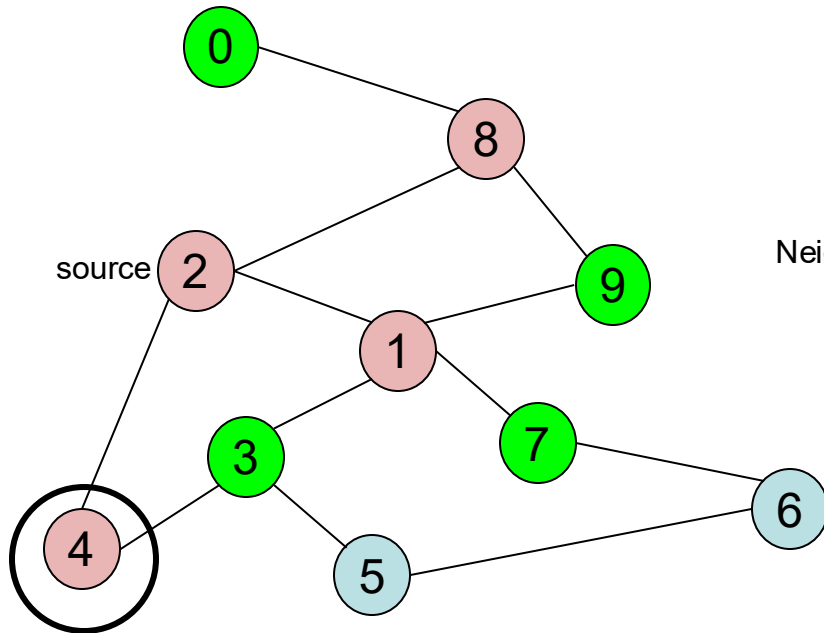
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

$Q = \{ 1, 4, 0, 9 \} \rightarrow Q = \{ 4, 0, 9, 3, 7 \}$



Adjacency List

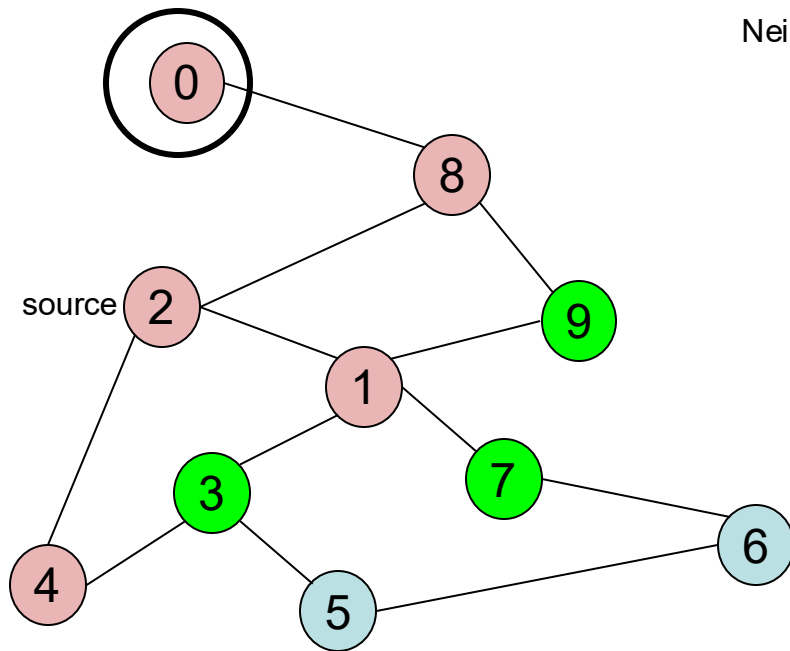
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Neighbors →

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

$\mathbf{Q} = \{ 4, 0, 9, 3, 7 \} \rightarrow \mathbf{Q} = \{ 0, 9, 3, 7 \}$



Neighbors →

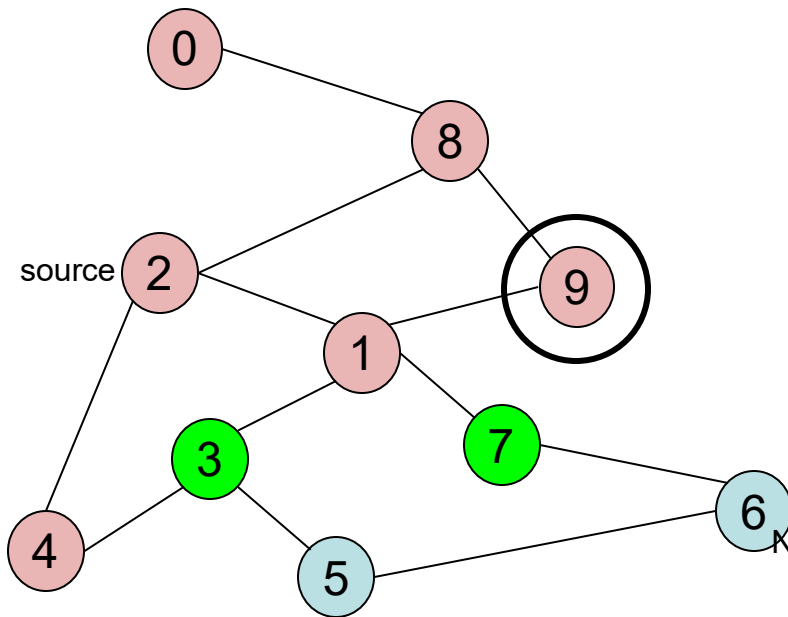
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

$\mathbf{Q} = \{ 0, 9, 3, 7 \} \rightarrow \mathbf{Q} = \{ 9, 3, 7 \}$



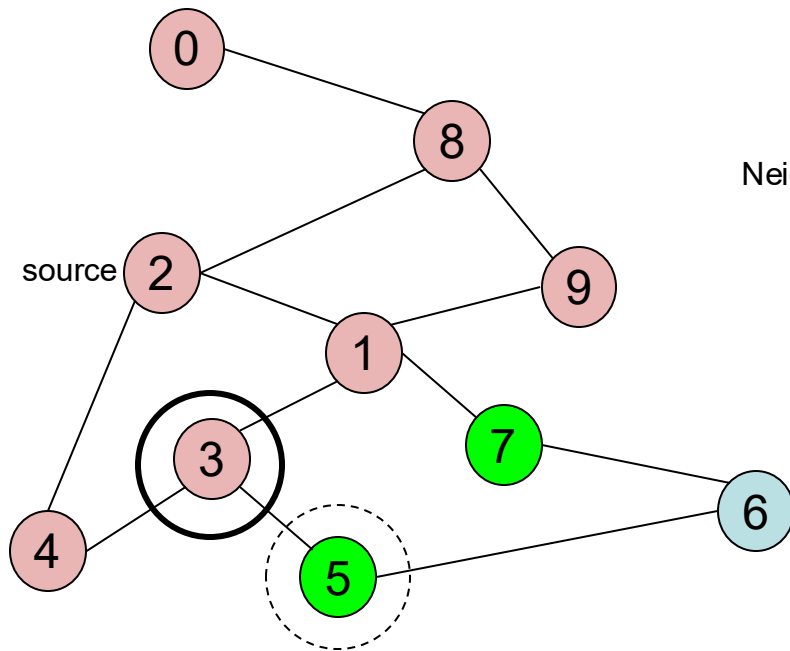
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

$Q = \{ 9, 3, 7 \} \rightarrow Q = \{ 3, 7 \}$



$Q = \{ 3, 7 \} \rightarrow Q = \{ 7, 5 \}$

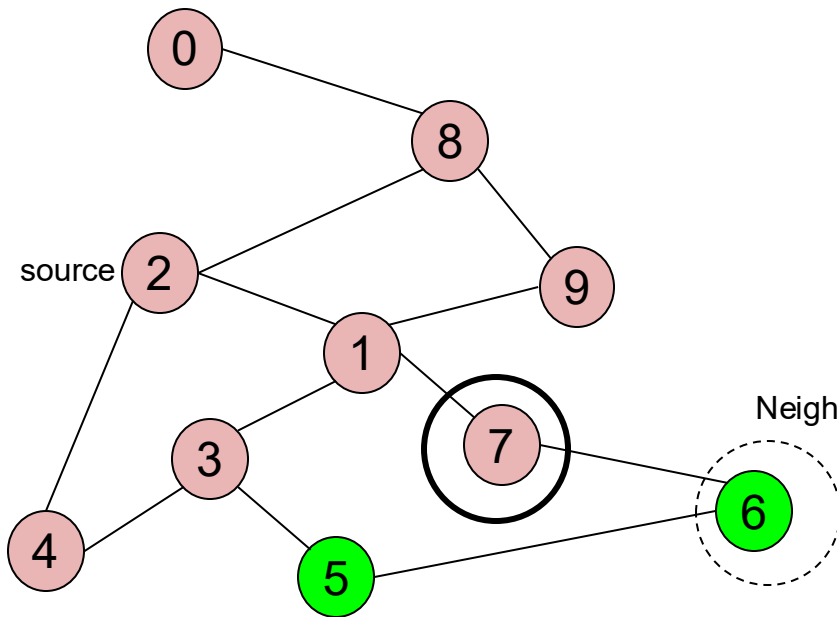
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Neighbors →

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	F
7	T
8	T
9	T



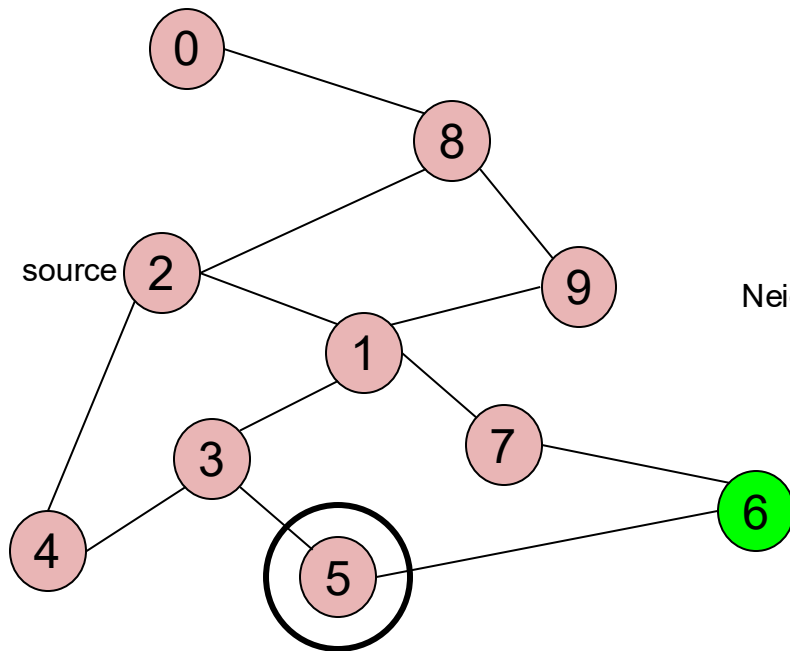
$Q = \{ 7, 5 \} \rightarrow Q = \{ 5, 6 \}$

Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T



Neighbors →

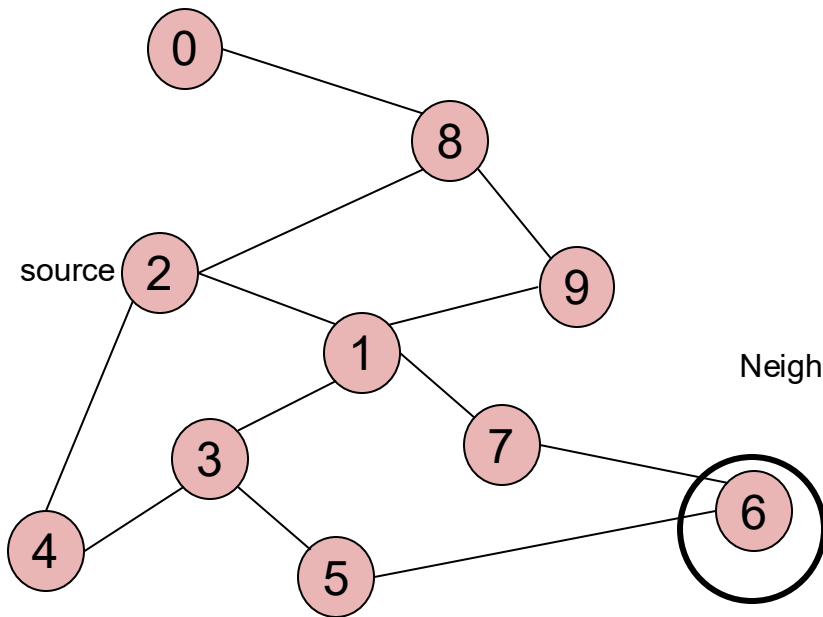
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

$Q = \{ 5, 6 \} \rightarrow Q = \{ 6 \}$



Neighbors →

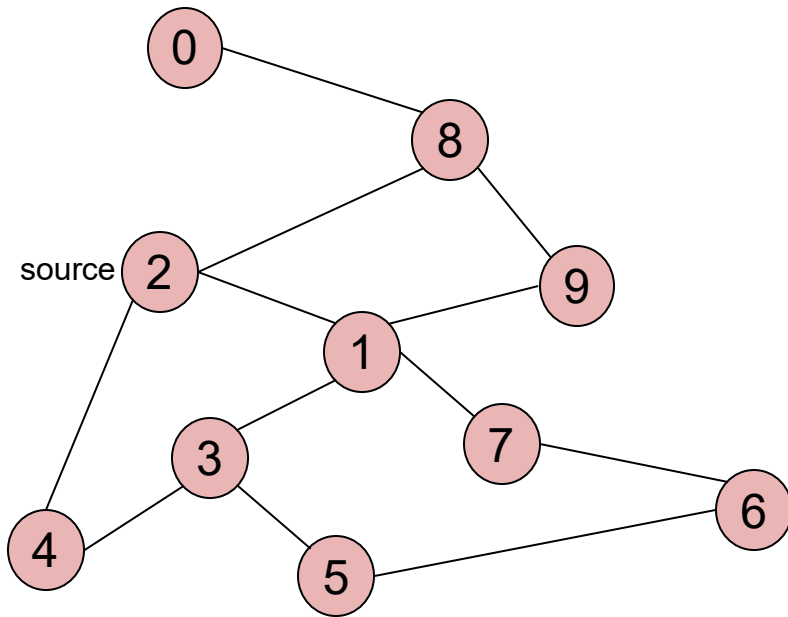
Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

$Q = \{ 6 \} \rightarrow Q = \{ \}$



Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

What did we discover?

Look at “visited” tables.

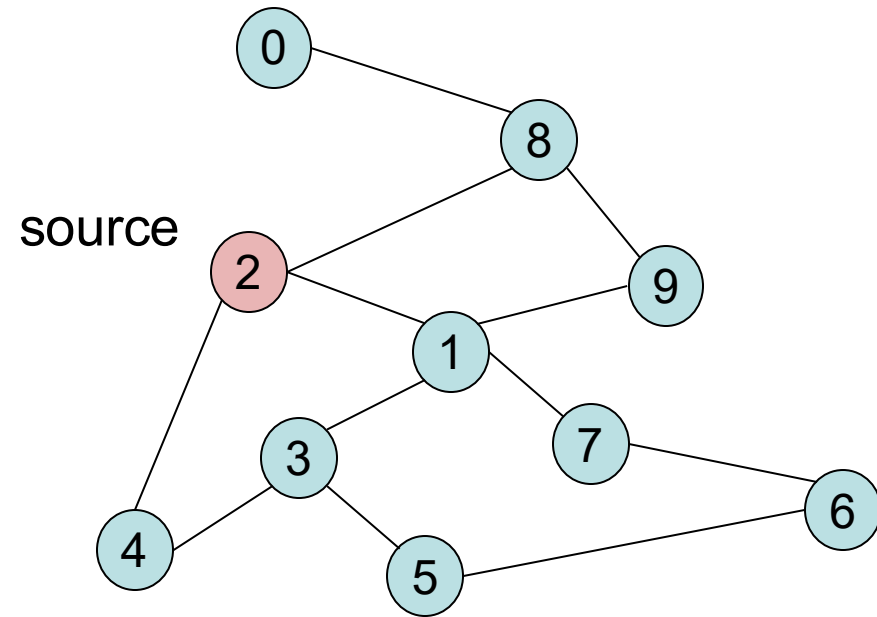
There exists a path from source vertex **2** to all vertices in the graph

Q = { } STOP!!! Q is empty!!!

Shortest Path Recording

- BFS only tells us whether a path exists from source s to other vertices v .
 - ♦ It doesn't tell us the path!
 - ♦ We need to *modify the algorithm to record the path*
- How can we do that?
 - ♦ We do not know which vertices lie on this path until we reach v !
 - ♦ But, for $\{w, v\}$, we know vertex w has been visited from vertex v .
 - ♦ Efficient solution:
 - *Use an additional array $pred[0..n-1]$ to record the predecessor for each vertex.*
 - *$Pred[w] = v$ means that vertex w has been visited *from* v .*

Example



$Q = \{ \quad \}$

Initialize **Q** to be empty

Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

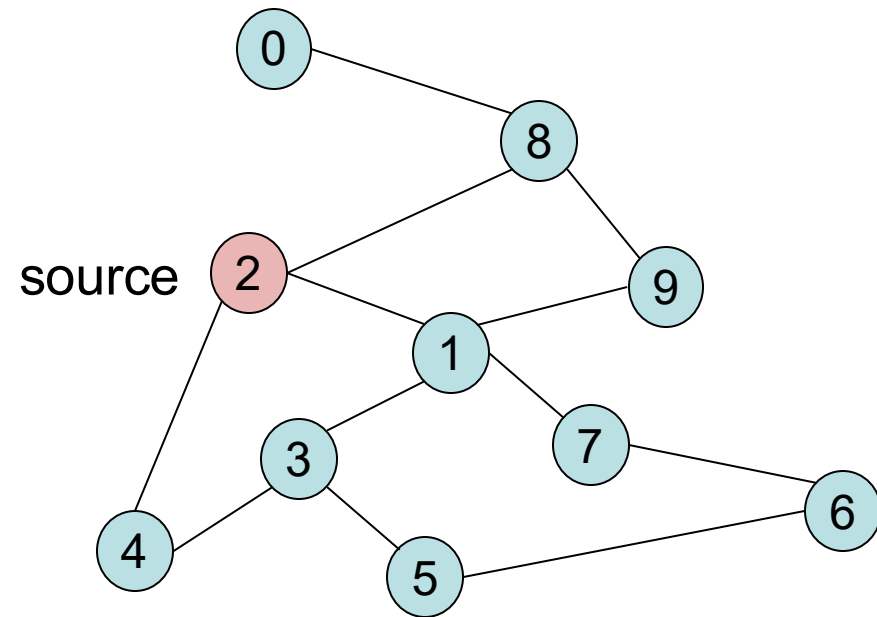
Visited Table (flag)
(T/F)

0	F	-
1	F	-
2	F	-
3	F	-
4	F	-
5	F	-
6	F	-
7	F	-
8	F	-
9	F	-

Pred

Initialize visited
table (all False)

Initialize **Pred** to -1



$Q = \{ 2 \}$

Adjacency List

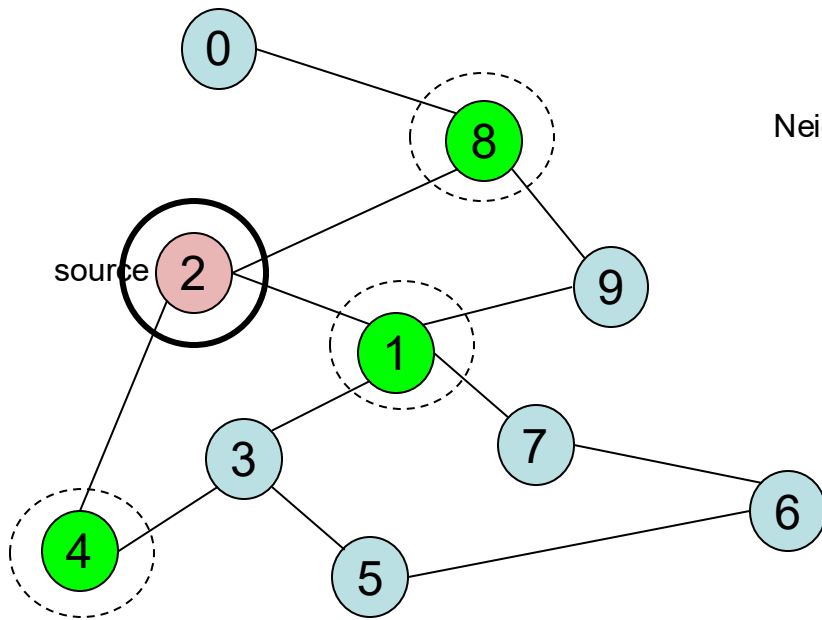
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	F
1	F
2	T
3	F
4	F
5	F
6	F
7	F
8	F
9	F

Pred

-
-
-
-
-
-
-
-
-
-



Neighbors →

Adjacency List

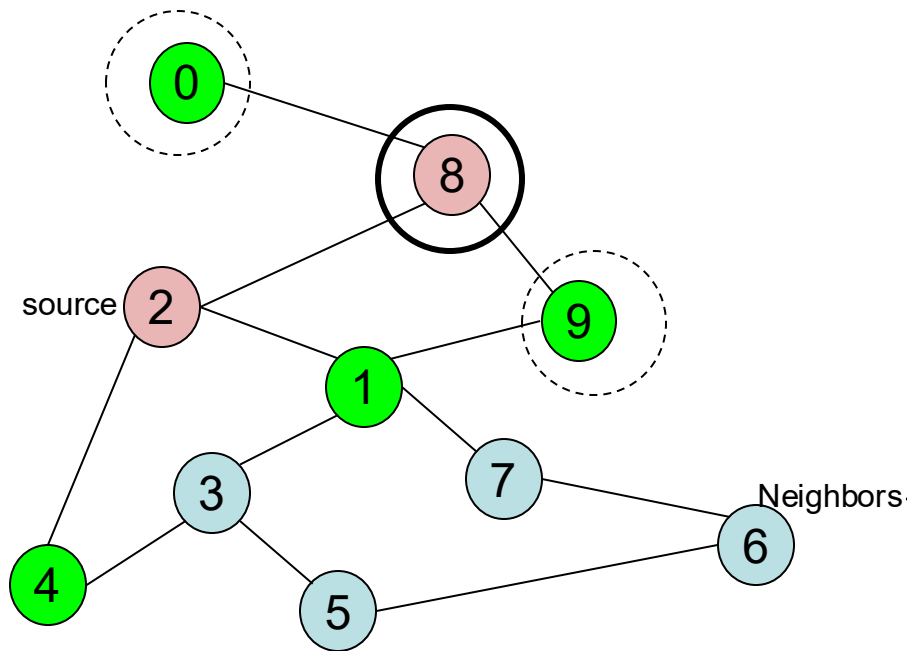
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	F
1	T
2	T
3	F
4	T
5	F
6	F
7	F
8	T
9	F

Pred

$Q = \{2\} \rightarrow Q = \{ 8, 1, 4 \}$



Adjacency List

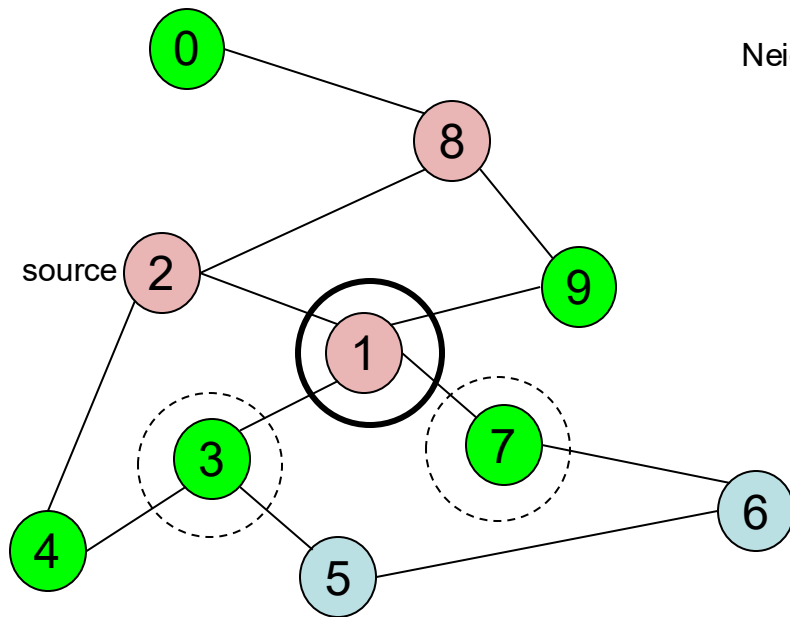
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T	8
1	T	2
2	T	-
3	F	-
4	T	2
5	F	-
6	F	-
7	F	-
8	T	2
9	T	8

Pred

$Q = \{ 8, 1, 4 \} \rightarrow Q = \{ 1, 4, 0, 9 \}$



Adjacency List

Neighbors →

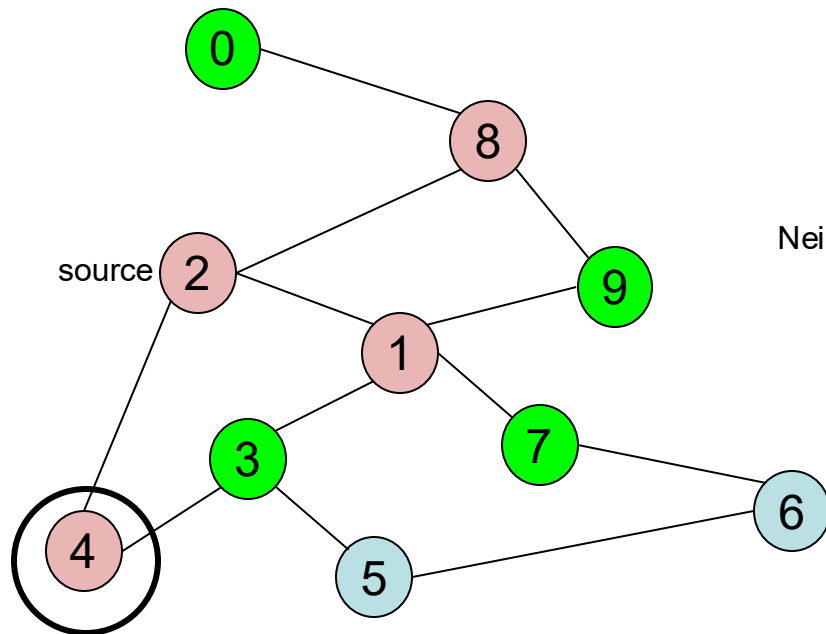
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T	8
1	T	2
2	T	-
3	T	1
4	T	2
5	F	-
6	F	-
7	T	1
8	T	2
9	T	8

Pred

$Q = \{ 1, 4, 0, 9 \} \rightarrow Q = \{ 4, 0, 9, 3, 7 \}$



Adjacency List

Neighbors →

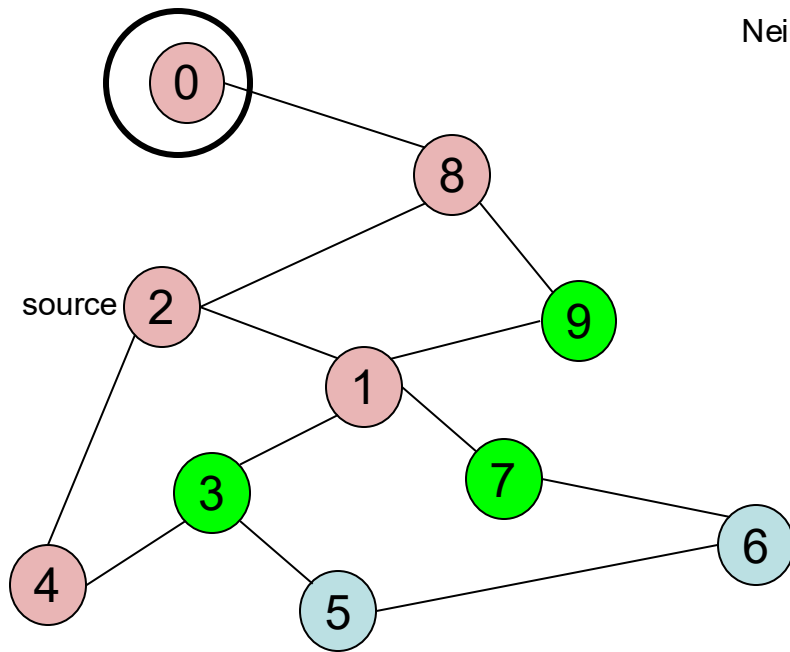
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T	8
1	T	2
2	T	-
3	T	1
4	T	2
5	F	-
6	F	-
7	T	1
8	T	2
9	T	8

Pred

$$Q = \{ 4, 0, 9, 3, 7 \} \rightarrow Q = \{ 0, 9, 3, 7 \}$$



Adjacency List

Neighbors →

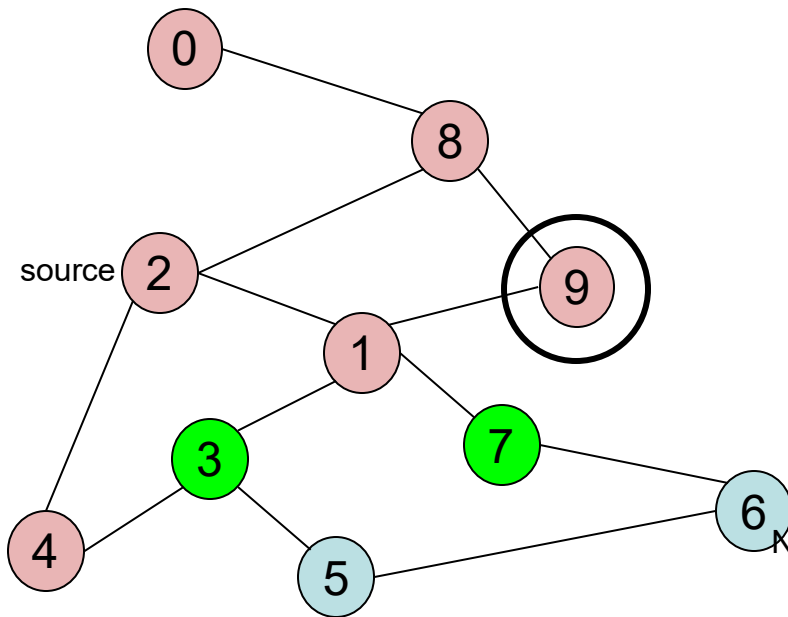
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

Pred

$Q = \{ 0, 9, 3, 7 \} \rightarrow Q = \{ 9, 3, 7 \}$



Adjacency List

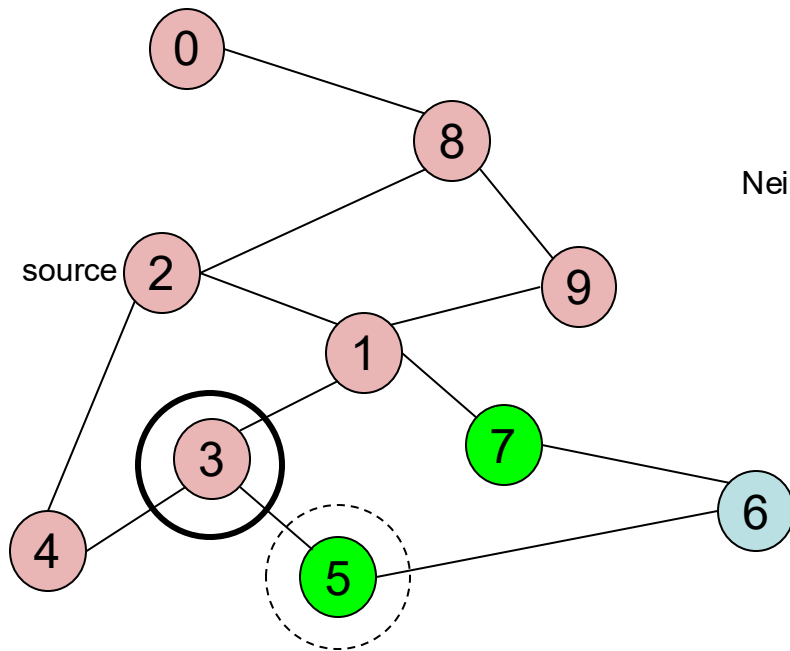
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	F
6	F
7	T
8	T
9	T

Pred

$\mathbf{Q} = \{ 9, 3, 7 \} \rightarrow \mathbf{Q} = \{ 3, 7 \}$



$Q = \{3, 7\} \rightarrow Q = \{7, 5\}$

Adjacency List

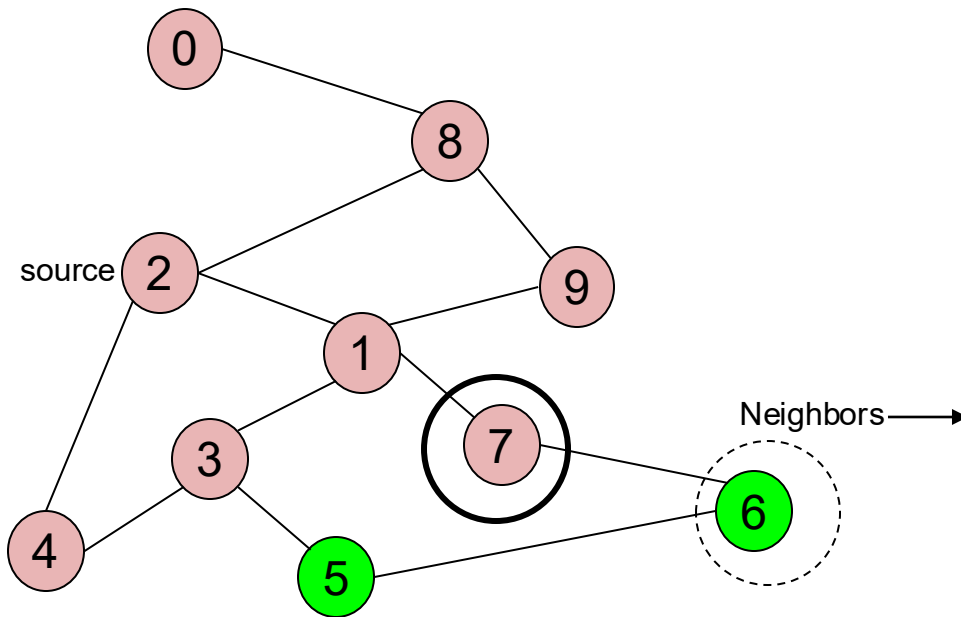
Neighbors →

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T	8
1	T	2
2	T	-
3	T	1
4	T	2
5	T	3
6	F	-
7	T	1
8	T	2
9	T	8

Pred



Adjacency List

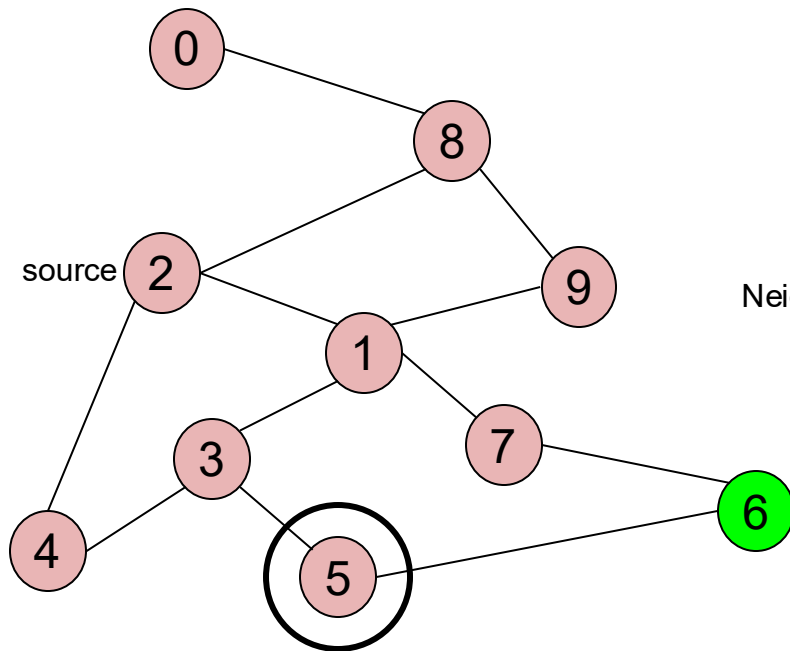
0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

Pred

$$Q = \{7, 5\} \rightarrow Q = \{5, 6\}$$



Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

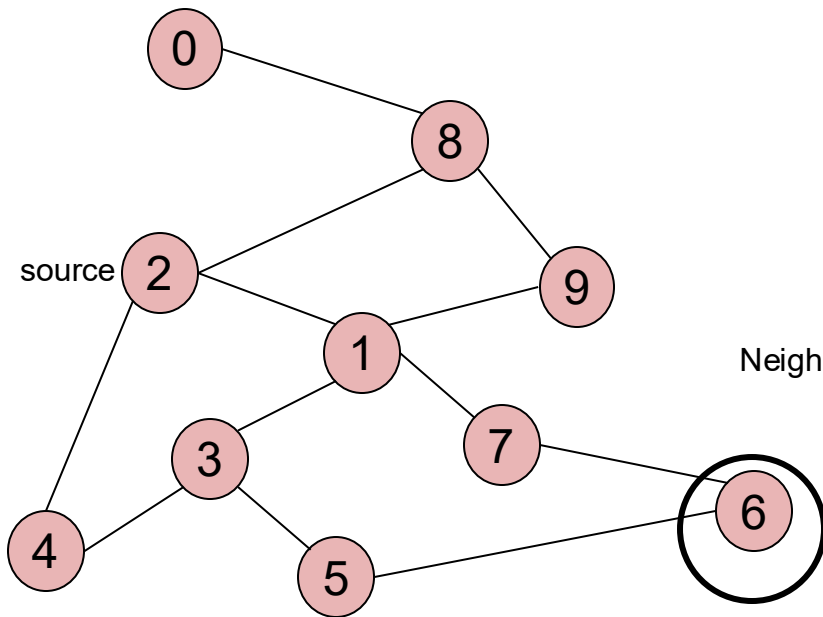
Neighbors →

Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

Pred

$\mathbf{Q} = \{ 5, 6 \} \rightarrow \mathbf{Q} = \{ 6 \}$



Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

Neighbors →

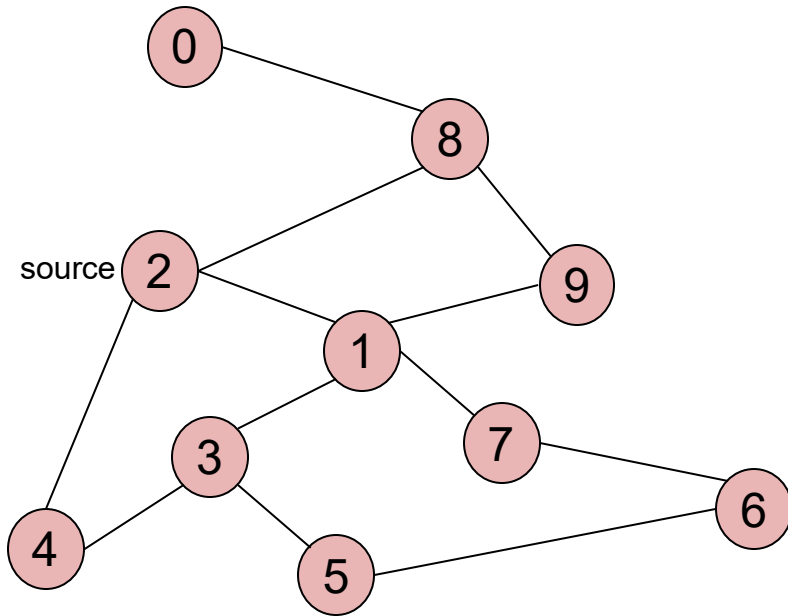
Visited Table (T/F)

0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T

Pred

$Q = \{ 6 \} \rightarrow Q = \{ \}$

BFS Finished



Adjacency List

0	8
1	3 7 9 2
2	8 1 4
3	4 5 1
4	2 3
5	3 6
6	7 5
7	1 6
8	2 0 9
9	1 8

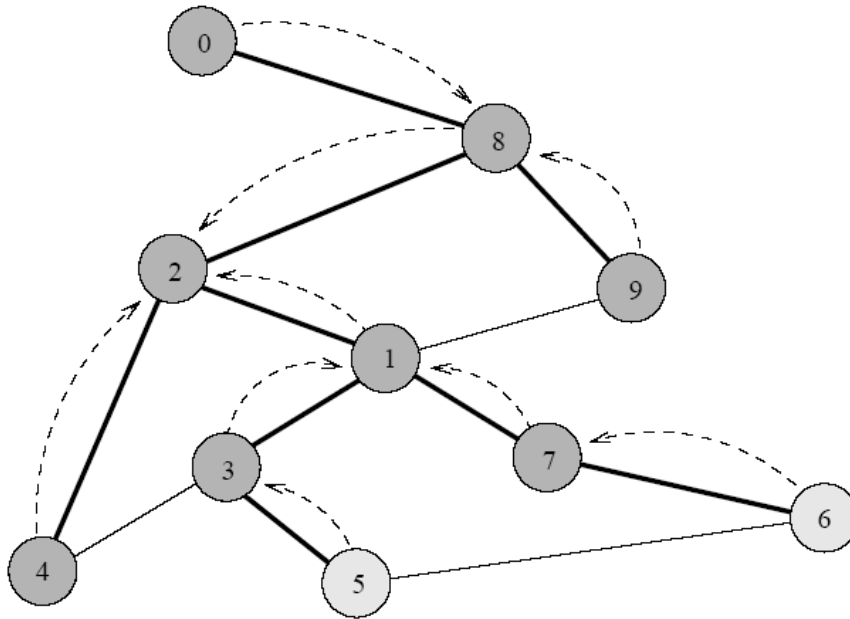
Visited Table (T/F)

0	T	8
1	T	2
2	T	-
3	T	1
4	T	2
5	T	3
6	T	7
7	T	1
8	T	2
9	T	8

Pred

Q = { } STOP!!! Q is empty!!!

Path Reporting



nodes	visited from
0	8
1	2
2	-
3	1
4	2
5	3
6	7
7	1
8	2
9	8

Recursive algorithm

Algorithm $Path(w)$

- ```

1. if $pred[w] \neq -1$
2. then
3. $Path(pred[w]);$
4. output w

```

Try some examples, report path from  $s$  to  $v$ :

Path(0)  $\rightarrow$  0- $\rightarrow$ 8- $\rightarrow$ 2

Path(6)  $\rightarrow$  6- $\rightarrow$ 7- $\rightarrow$ 1- $\rightarrow$ 2

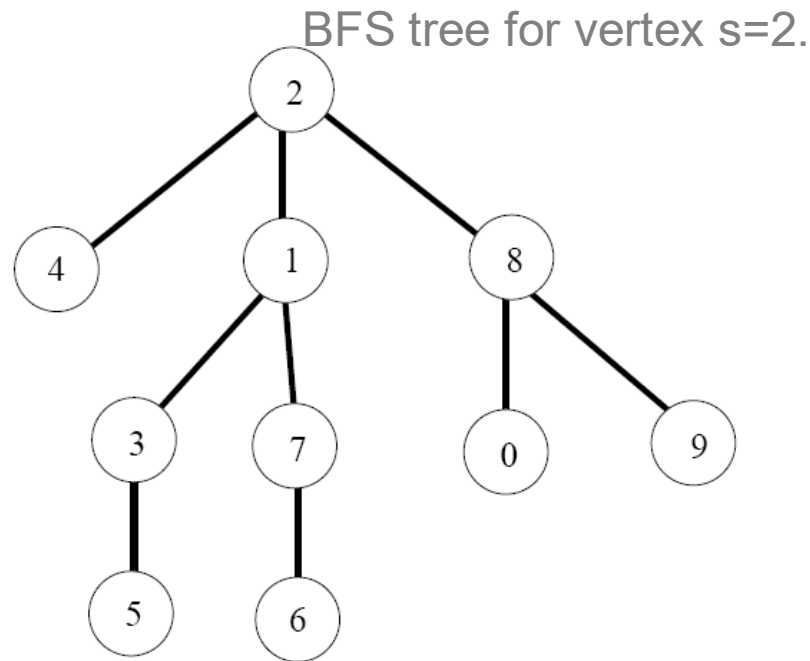
Path(1)  $\rightarrow$  1- $\rightarrow$ 2

The path returned is the shortest from s to v (minimum number of edges).



# BFS Tree

- The paths found by BFS is often drawn as a rooted tree (called **BFS tree**, with the **starting vertex as the root** of the tree).



nodes

|   |   |
|---|---|
| 0 | 8 |
| 1 | 2 |
| 2 | - |
| 3 | 1 |
| 4 | 2 |
| 5 | 3 |
| 6 | 7 |
| 7 | 1 |
| 8 | 2 |
| 9 | 8 |

---

# Depth-First Search

# Depth-First Search

- DFS is another popular graph searching or traversal strategy.
  - ♦ Idea is similar to [pre-order traversal](#) (visit node, then visit children recursively).
  - ♦ Process [all of a vertex's descendents](#) before moving to an adjacent vertex.

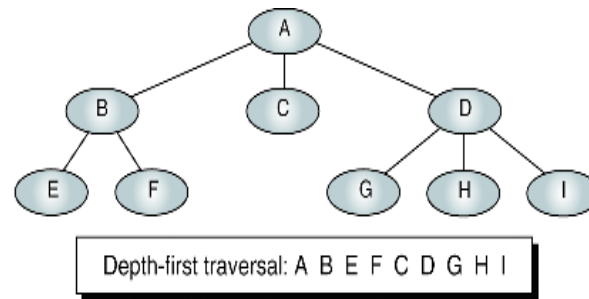
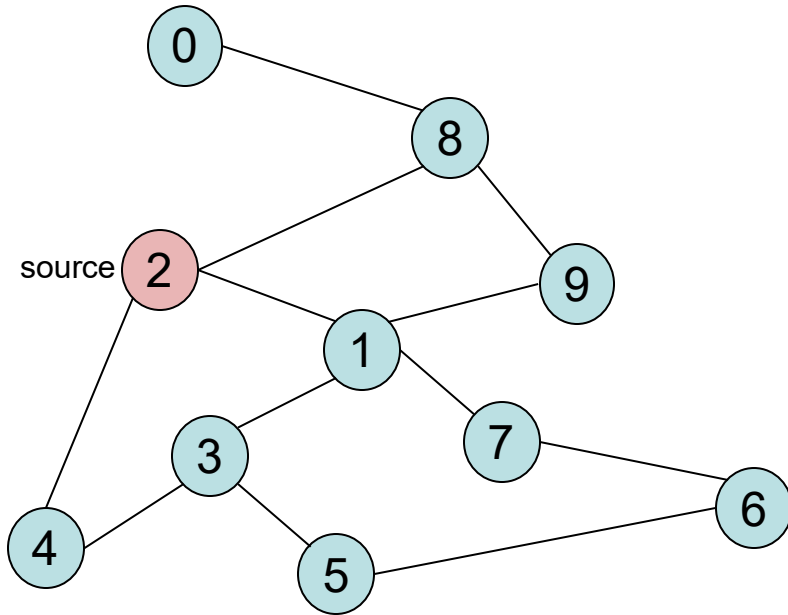


FIGURE 11-9 Depth-first Traversal of a Tree

# Example



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

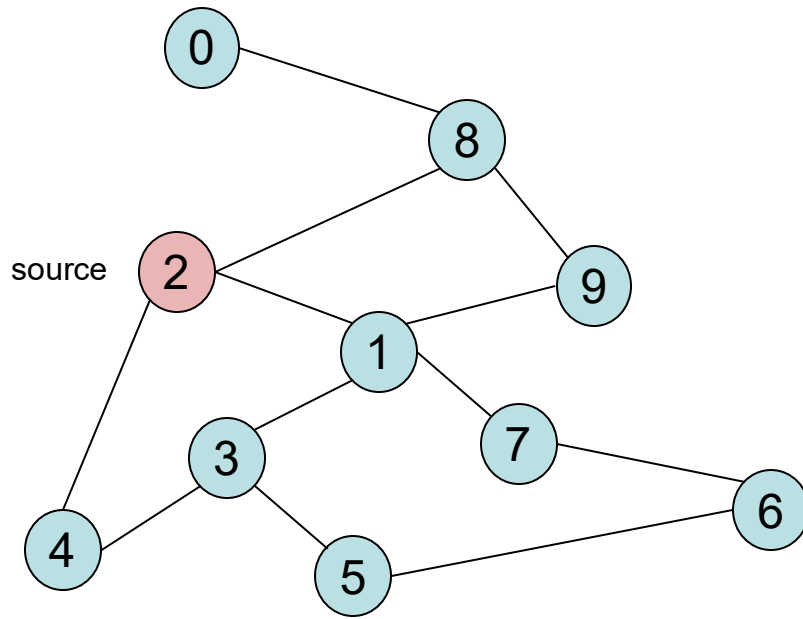
|   |   |
|---|---|
| 0 | F |
| 1 | F |
| 2 | F |
| 3 | F |
| 4 | F |
| 5 | F |
| 6 | F |
| 7 | F |
| 8 | F |
| 9 | F |

*Pred*

|   |
|---|
| - |
| - |
| - |
| - |
| - |
| - |
| - |
| - |
| - |
| - |

Initialize visited  
table (all False)

Initialize Pred to -1



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | F | - |
| 1 | F | - |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | F | - |
| 9 | F | - |

*Pred*

RDFS( 2 )

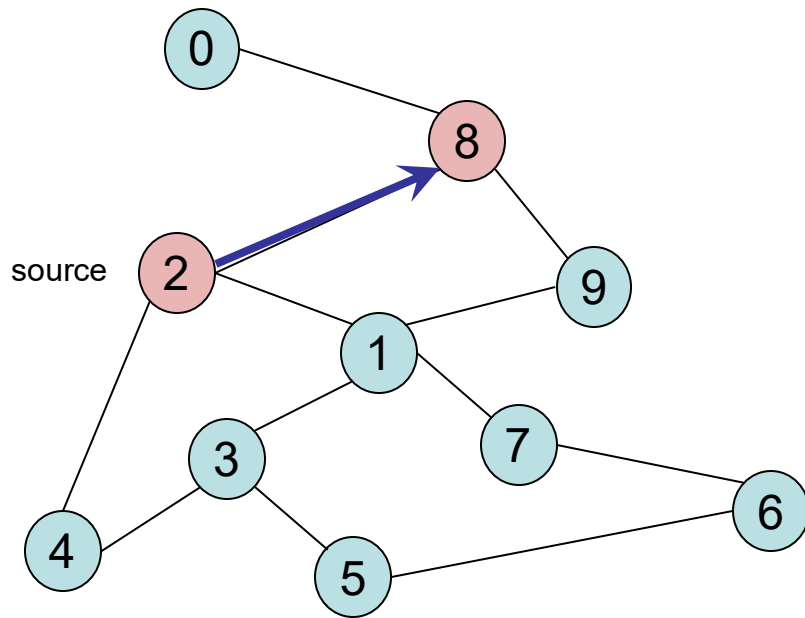
Now visit RDFS(8)

**Algorithm**  $DFS(s)$

1. **for** each vertex  $v$
2.     **do**  $flag[v] := \text{false};$
3.      $RDFS(s);$

**Algorithm**  $RDFS(v)$

1.  $flag[v] := \text{true};$
2. **for** each neighbor  $w$  of  $v$
3.     **do if**  $flag[w] = \text{false}$
4.         **then**  $RDFS(w);$



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | F | - |
| 1 | F | - |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | F | - |

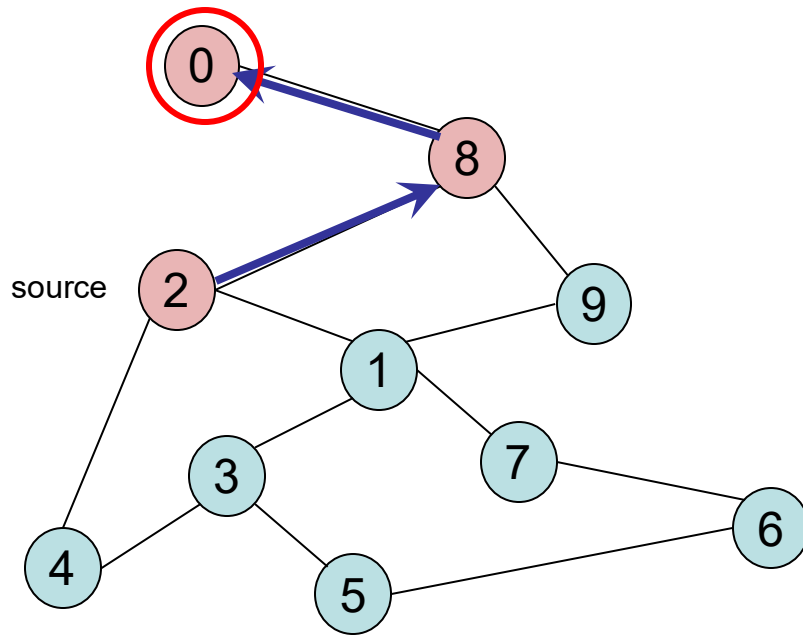
*Pred*

Recursive  
calls

RDFS( 2 )

RDFS(8)

2 is already visited, so visit RDFS(0)



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | F | - |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | F | - |

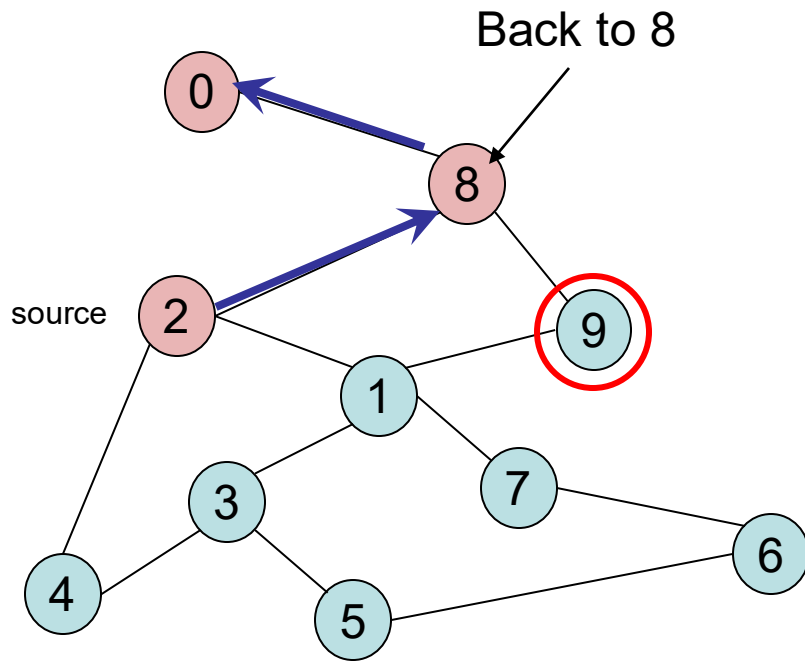
*Pred*

Recursive  
calls

RDFS( 2 )

RDFS(8)

RDFS(0) -> no unvisited neighbors, return  
to call RDFS(8)



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | F | - |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | F | - |

*Pred*

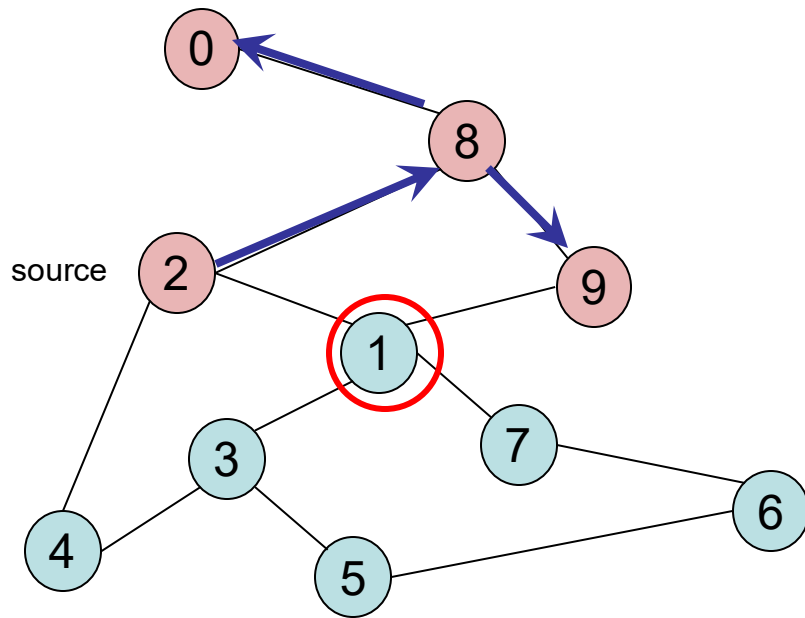
Recursive  
calls

RDFS( 2 )

RDFS(8)

Now visit 9 -> RDFS(9)





Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

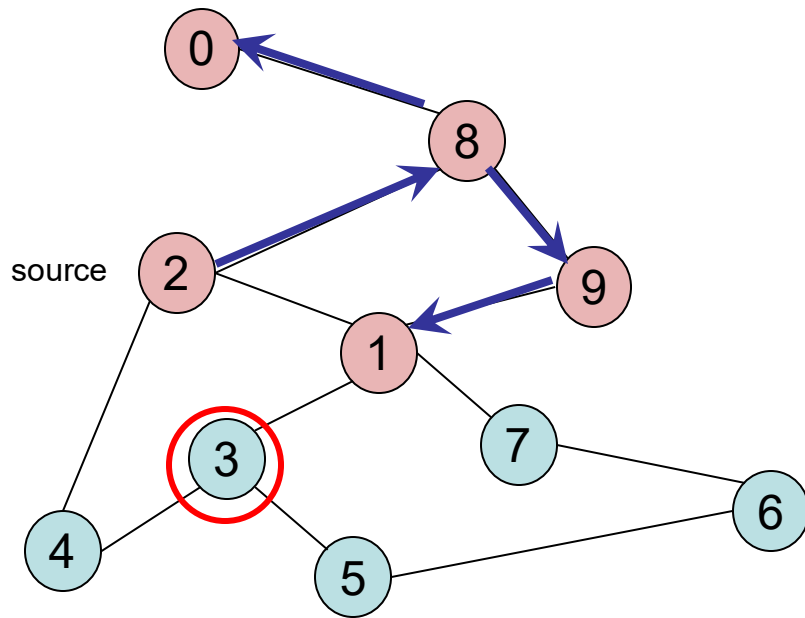
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | F | - |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Recursive  
calls

RDFS( 2 )  
 RDFS(8)  
 RDFS(9)  
 -> visit 1, RDFS(1)



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

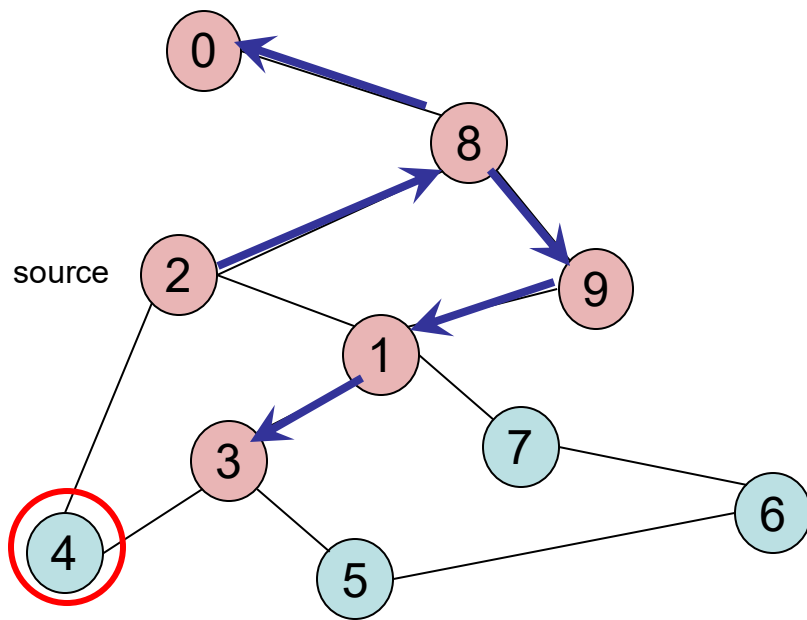
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | F | - |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Recursive calls

RDFS( 2 )  
 RDFS(8)  
 RDFS(9)  
 RDFS(1)  
 visit RDFS(3)



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

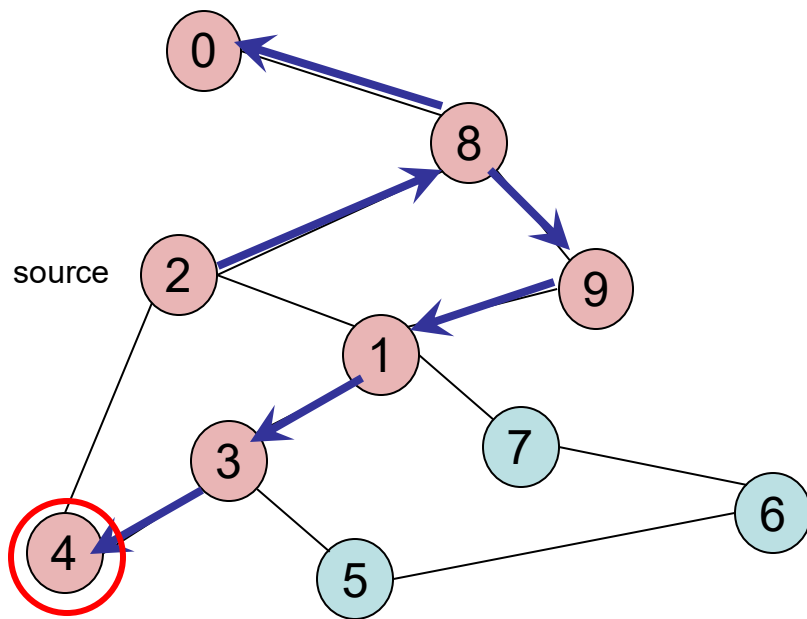
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | F | - |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Recursive  
calls

RDFS( 2 )  
   RDFS(8)  
     RDFS(9)  
       RDFS(1)  
         RDFS(3)  
           visit RDFS(4)



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )

RDFS(8)

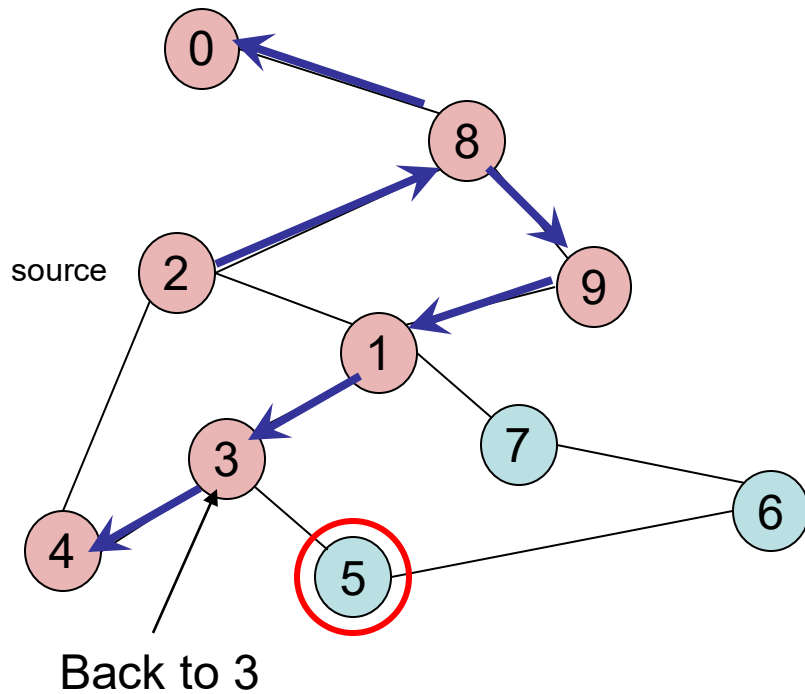
RDFS(9)

RDFS(1)

RDFS(3)

RDFS(4) → STOP all of 4's neighbors have been visited  
return back to call RDFS(3)

Recursive  
calls



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | F | - |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )

RDFS(8)

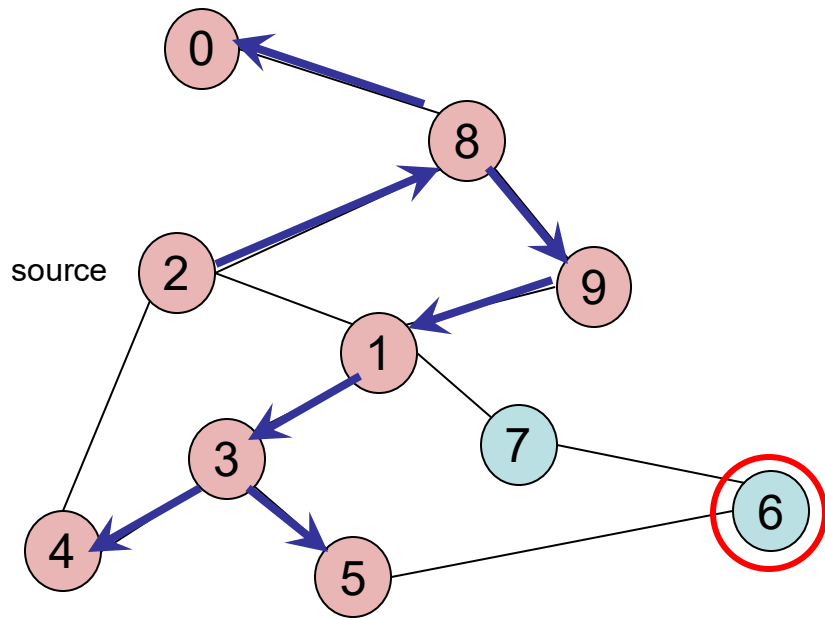
RDFS(9)

RDFS(1)

RDFS(3)

visit 5 -> RDFS(5)

Recursive  
calls



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | F | - |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )

RDFS(8)

RDFS(9)

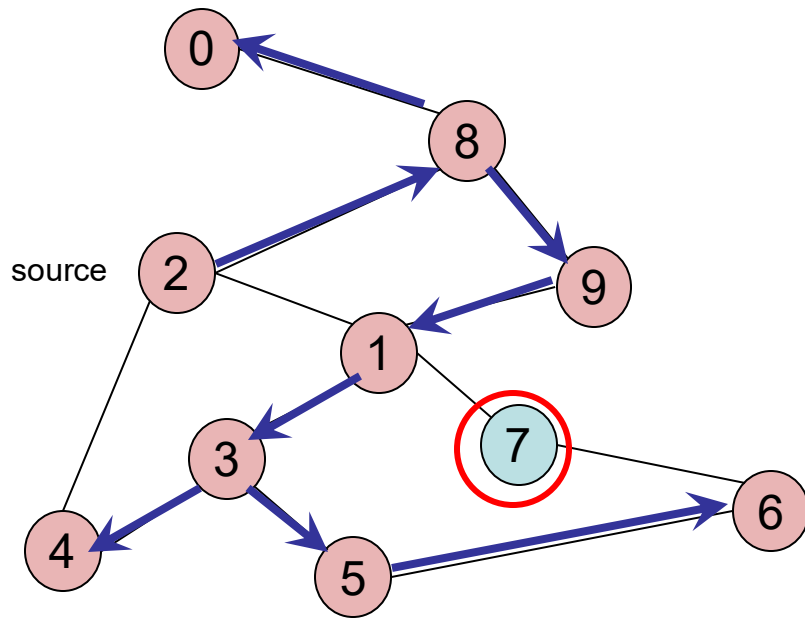
RDFS(1)

RDFS(3)

RDFS(5)

3 is already visited, so visit 6 -> RDFS(6)

Recursive  
calls



RDFS( 2 )

RDFS(8)

RDFS(9)

RDFS(1)

RDFS(3)

RDFS(5)

RDFS(6)

visit 7 -> RDFS(7)

Adjacency List

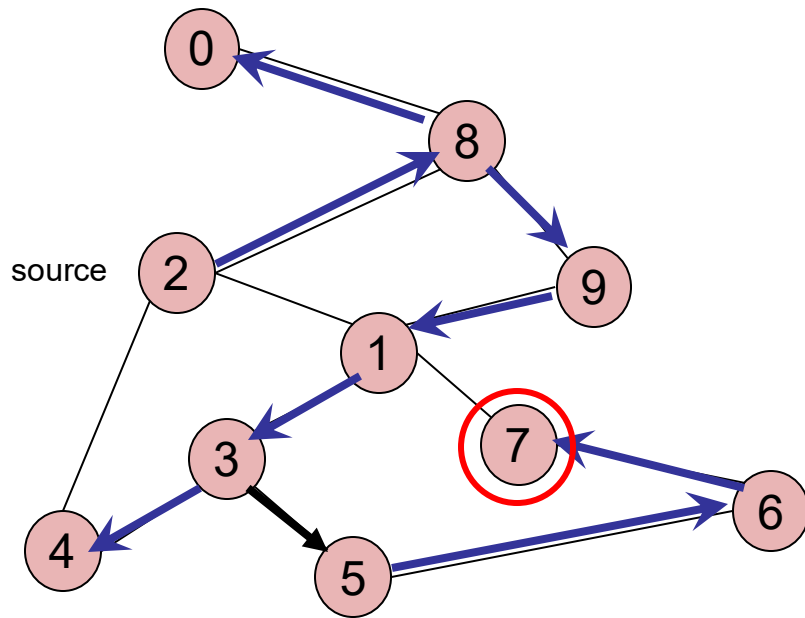
|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | F | - |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Recursive  
calls



RDFS( 2 )

RDFS(8)

RDFS(9)

RDFS(1)

RDFS(3)

RDFS(5)

RDFS(6)

RDFS(7) -> Stop no more unvisited neighbors

Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

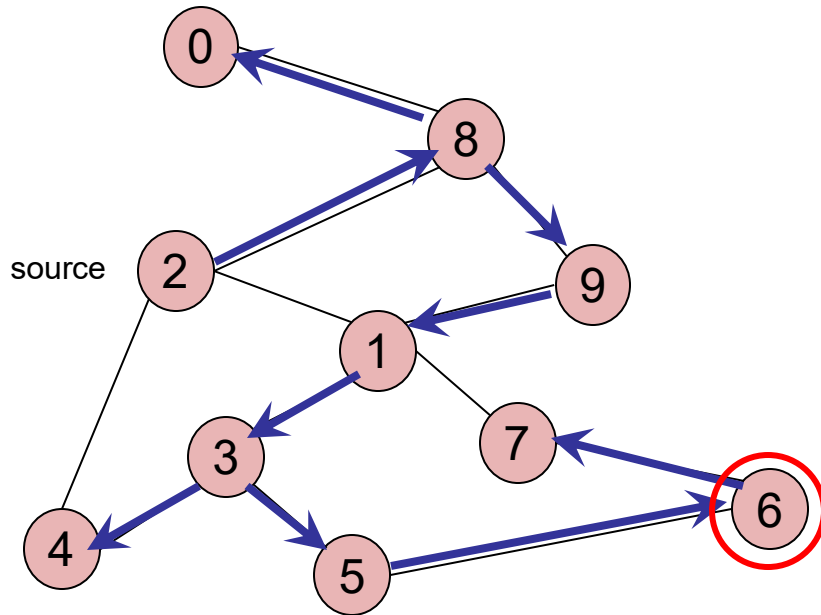
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Recursive  
calls





Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )

RDFS(8)

RDFS(9)

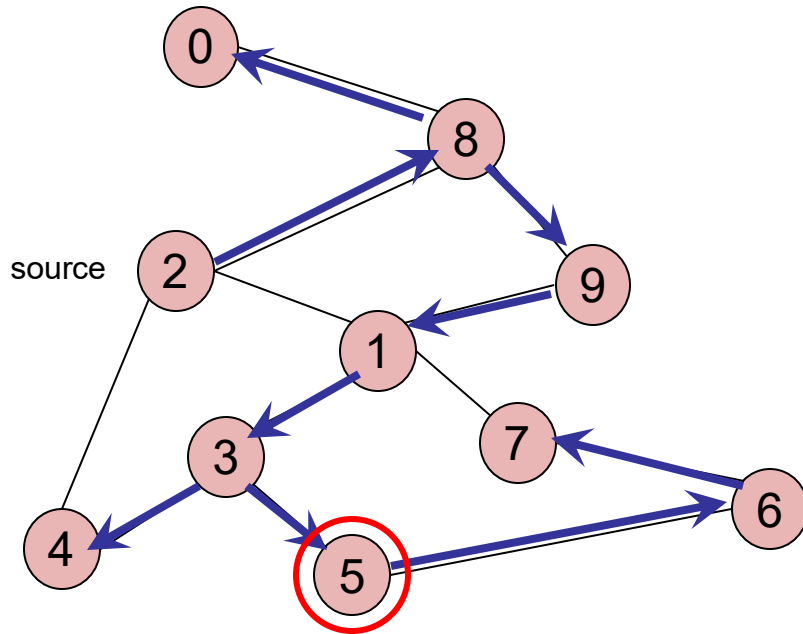
RDFS(1)

RDFS(3)

RDFS(5)

RDFS(6) -> Stop

Recursive  
calls



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

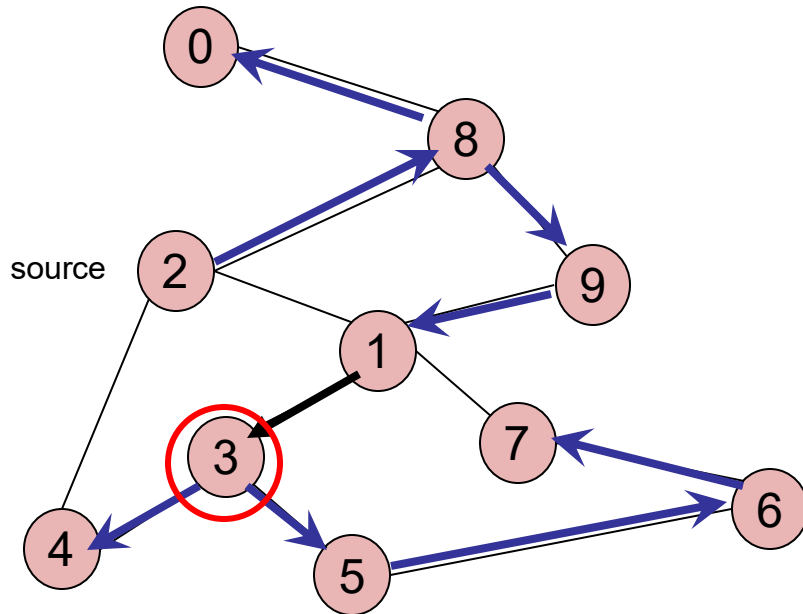
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )  
   RDFS(8)  
     RDFS(9)  
       RDFS(1)  
         RDFS(3)  
           RDFS(5) -> Stop

Recursive  
calls



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

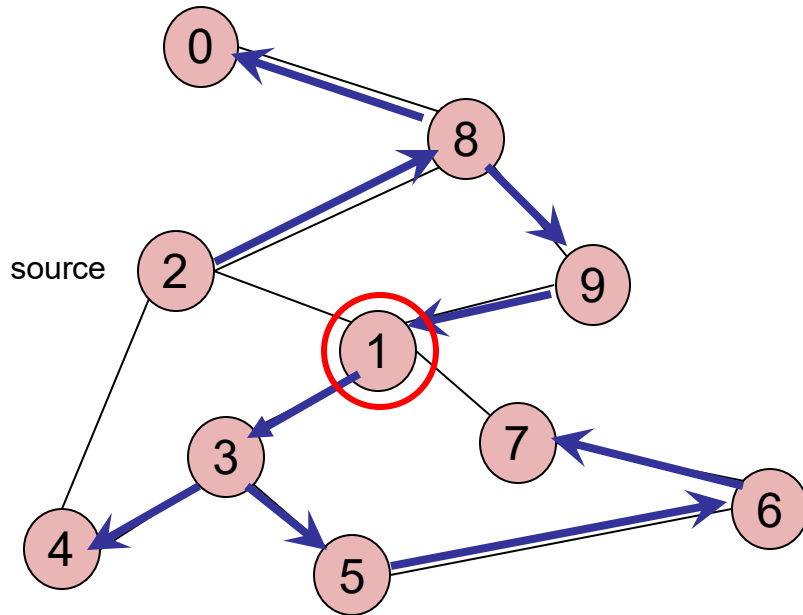
Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

RDFS( 2 )  
   RDFS(8)  
     RDFS(9)  
       RDFS(1)  
         RDFS(3) -> Stop

Recursive  
calls



RDFS( 2 )

RDFS(8)

RDFS(9)

RDFS(1) -> Stop

Recursive  
calls

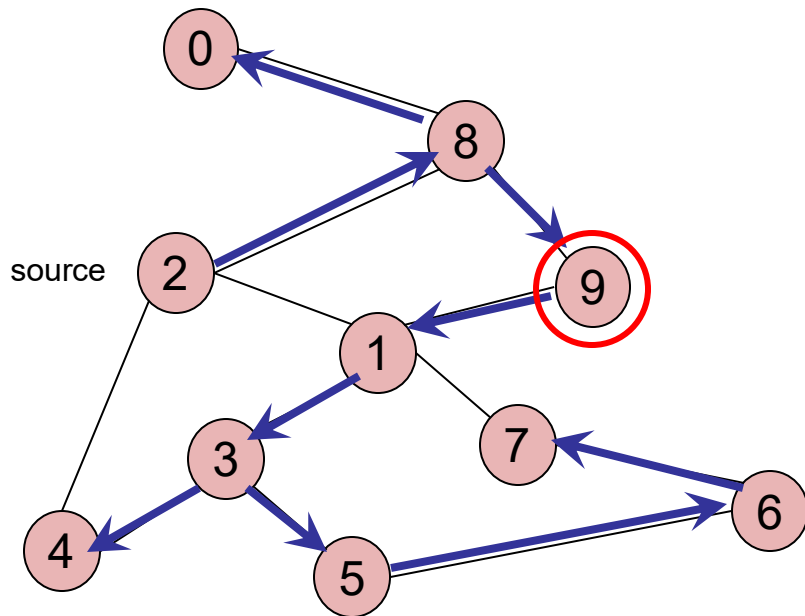
Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*



Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

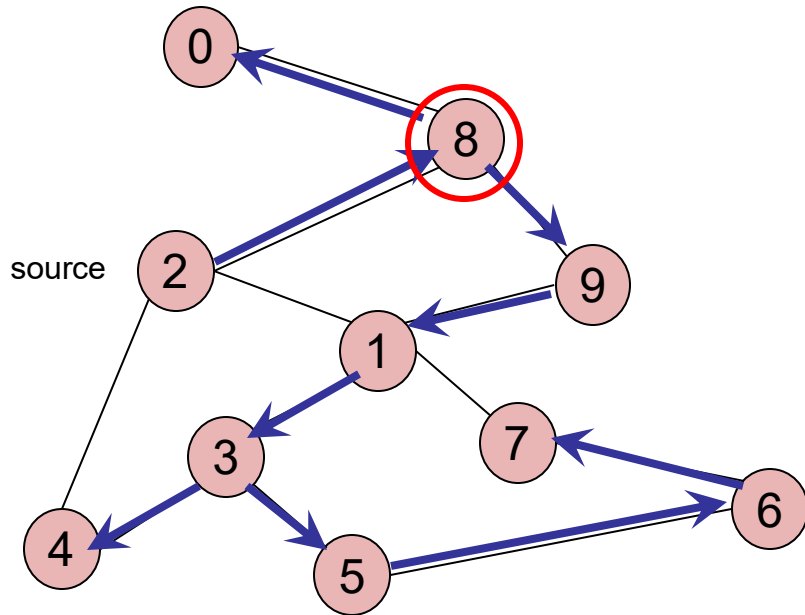
*Pred*

RDFS( 2 )

RDFS(8)

RDFS(9) -> Stop

Recursive  
calls



RDFS( 2 )  
RDFS(8) -> Stop

Recursive  
calls

Adjacency List

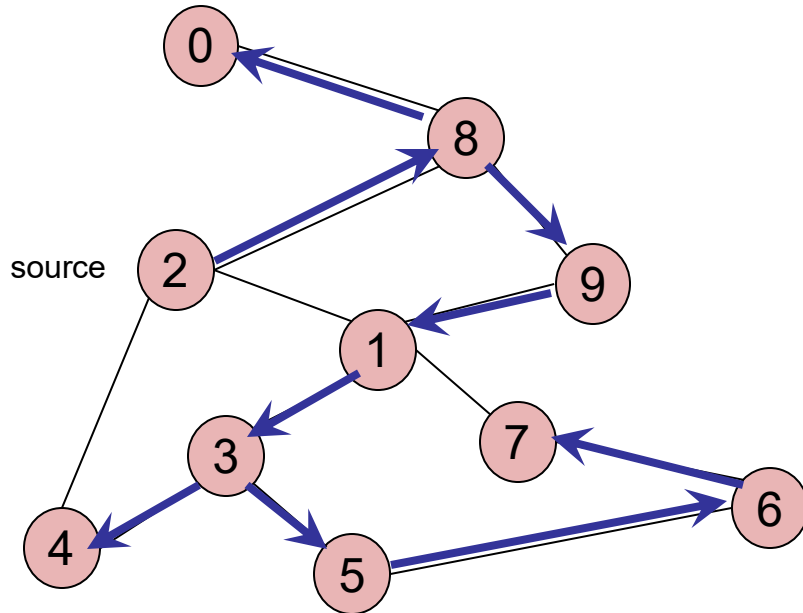
|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

# Example Finished



RDFS( 2 ) -> Stop

Recursive calls finished

Adjacency List

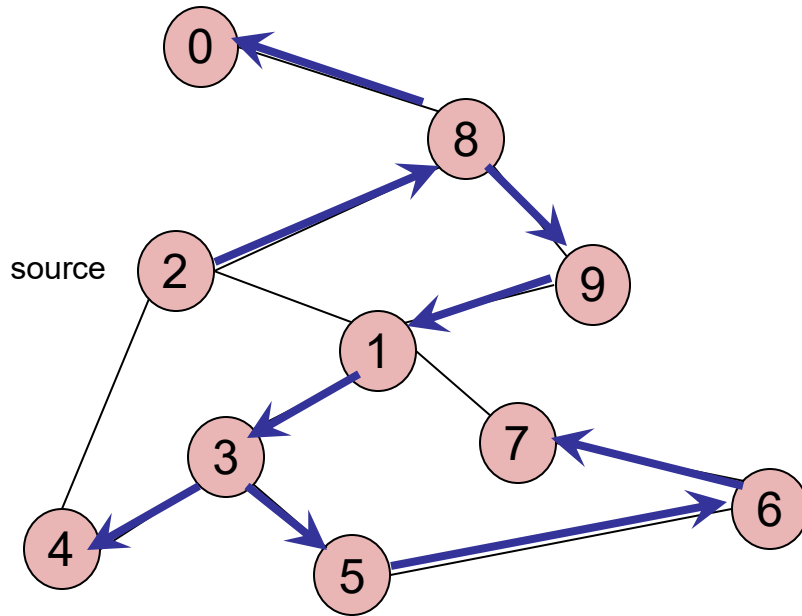
|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

# DFS Path Tracking



DFS finds out path too

**Algorithm**  $Path(w)$

1. **if**  $pred[w] \neq -1$
2.     **then**
3.          $Path(pred[w]);$
4.     output  $w$

Adjacency List

|   |         |
|---|---------|
| 0 | 8       |
| 1 | 3 7 9 2 |
| 2 | 8 1 4   |
| 3 | 4 5 1   |
| 4 | 2 3     |
| 5 | 3 6     |
| 6 | 7 5     |
| 7 | 1 6     |
| 8 | 2 0 9   |
| 9 | 1 8     |

Visited Table (T/F)

|   |   |   |
|---|---|---|
| 0 | T | 8 |
| 1 | T | 9 |
| 2 | T | - |
| 3 | T | 1 |
| 4 | T | 3 |
| 5 | T | 3 |
| 6 | T | 5 |
| 7 | T | 6 |
| 8 | T | 2 |
| 9 | T | 8 |

*Pred*

Try some examples.

$Path(0) \rightarrow 0-8-2$

$Path(6) \rightarrow 6-5-3-1-9-8-2$

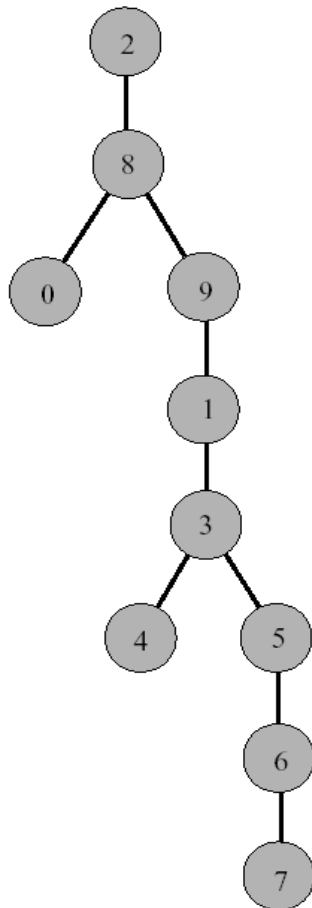
$Path(7) \rightarrow 7-6-5-3-1-9-8-2$



# DFS Tree

Resulting DFS-tree.

Notice it is much “**deeper**” than the BFS-tree.



|   |   |
|---|---|
| 0 | T |
| 1 | T |
| 2 | T |
| 3 | T |
| 4 | T |
| 5 | T |
| 6 | T |
| 7 | T |
| 8 | T |
| 9 | T |

|   |
|---|
| 8 |
| 9 |
| - |
| 1 |
| 3 |
| 3 |
| 5 |
| 6 |
| 2 |
| 8 |

*Pred*



# Summary

- Graph traversal
  - ♦ Breadth-First Search
    - By queue
  - ♦ Depth-First Search
    - By stack (non-recursively)
    - Recursively
- Spanning tree
  - ♦ BFS tree
  - ♦ DFS tree